

labeled data and β -weak-labeled data is then incorporated into the training mixture. To state this procedure formally, it will be convenient to introduce the **generate** subroutine, which issues multiple calls to an LLM per prompt to produce a dataset of synthetically labeled prompts.

Definition 4 (Generation). The function **generate** takes as input a prompt set $P \subseteq \mathcal{X}$, LLM g and positive integer k , and is defined

$$\text{generate}(P; k, g) = \{(x, y_x^i) : x \in P, i \in [k], y_x^i \sim g(x)\}.$$

Algorithm 2 formalizes our procedure for data generation, where generation, filtering, and weak-labeling are applied in sequence on each generation of LLM. Whether the data that is being added to the mixture consists of mostly β -weakly labeled data (D_t in Algorithm 2) or γ -filtered synthetic data (S_t^+ in Algorithm 2) is parameterized by $\alpha > 0$.

Algorithm 2 Boosting-style algorithm for LLM training

Given: Prompt set P , repeat parameter k , weakly labeled prompt weight α , high-quality fraction β , filter recall γ , number of iterations T .

- 1: $g_0 = \perp$ and $\mathcal{D}_0 = \emptyset$ ▷ Initial LLM and initial training set
 - 2: **for** $t = 1, 2, \dots, T$ **do**
 - 3: $S_t = \text{generate}(P; k, g_{t-1})$ ▷ Generate k synthetic responses per prompt.
 - 4: $(S_t^+, P_t^-) = \text{filter}_\gamma(S_t)$ ▷ Noisily partition high-quality synthetic data from low-quality prompts.
 - 5: $D_t = \text{labeler}_\beta(P_t^-)$ ▷ Weakly label low-quality prompts
 - 6: $\lambda_t = \frac{\alpha}{|D_t|}$ ▷ Set weight of weakly labeled prompts
 - 7: $\mathcal{D}_t = \mathcal{D}_{t-1} \uplus \lambda_t D_t \uplus S_t^+$ ▷ Update training mixture
 - 8: $g_t = \text{learner}(\mathcal{D}_t)$ ▷ Use learner to update LLM
 - 9: **end for**
-

6 Main result

Theorem 5 is our main theoretical result, and states that the final LLM g_T output by Algorithm 2 satisfies the convergence requirement in Eq. (1). Theorem 5 also quantifies the rate of convergence.

Theorem 5. *Let $\varepsilon \in (0, 1)$. Suppose that in Algorithm 2 we have $\alpha > 0$, $\beta \in (0, 1)$, $\gamma \in (0, 1]$*

$$T \geq \frac{\log(2/\varepsilon)}{\beta} + \frac{2\alpha}{\beta\varepsilon} + 1$$

and $k \geq (2\log T + \log |P|)/(\beta\gamma)$. With probability at least $1 - 1/T$ over the randomness of the algorithm, the final LLM g_T output by the algorithm satisfies

$$\Pr_{x \sim P, y \sim g_T(x)}[q(x, y) = 1] \geq 1 - \varepsilon.$$

Note that by setting $\alpha = \varepsilon$ in Algorithm 2 the iteration complexity becomes $T = O(\log(1/\varepsilon)/\beta)$.

Proof sketch. The key step in the proof is showing that, with probability $1 - 1/T$, in each iteration t we have $\Pr_{y \sim g_{t-1}(x)}[q(x, y) = 1] \geq \beta$ for all but $(1 - \beta)^{t-1}$ fraction of the prompts $x \in P$. Since

the algorithm draws $k = \Omega(1/(\beta\gamma))$ synthetic responses to each prompt from g_{t-1} , one of those responses is likely to be correct. As a result, correctly labeled prompts are continually added to the training data (via the synthetic dataset S_t^+), and the quality of the training data steadily improves, causing the performance of the LLMs learned from that training data to approach the optimal performance.

Even when t is large, it is non-trivial to show that $\Pr_{y \sim g_{t-1}(x)}[q(x, y) = 1] \geq \beta$ for nearly all prompts $x \in P$. While this fact follows from our assumption about the weak labeler, it does not follow straightforwardly. The weak labeler ensures that the *average* response quality to a given set of prompts is at least β , but we need a guarantee about response quality that holds *uniformly* for almost all prompts. Our approach is to first show that P_t^- (the set of prompts with low-quality responses) shrinks exponentially with t , and then observe that the total weight assigned to these prompts in the training data is fixed at $\alpha > 0$ (a free parameter of our algorithm). Consequently, once a prompt is assigned a high-quality response by the weak labeler, the weight of that response overwhelms the weight of all previous low-quality responses in the training data. So when the learner fits an LLM to this training data, the LLM assigns non-trivial probability mass to the high-quality response; we are able to bound this probability from below by β . $\square$

6.1 Relationship to Boosting

Boosting is a meta-learning algorithm for combining weak hypotheses into highly accurate ensemble classifiers [Schapire and Freund, 2013]. While the most common version of boosting is AdaBoost [Freund and Schapire, 1997], we will present a slightly simpler version that still contains all of the essential ideas.

In each iteration of boosting, a training set of binary-labeled examples is given as input to a *weak learner*. Each training example is associated with a non-negative weight, and the weights sum to 1. The weak learner returns a hypothesis that achieves weighted error at most $\frac{1}{2} - \beta$ on the training set, where $\beta \in (0, \frac{1}{2})$ is the *edge* over the trivial hypothesis that randomly guesses each label. The weight on each training example that is correctly labeled by the hypothesis is decreased by a factor $\exp(\theta)$, and the weight on each training example that is incorrectly labeled by the hypothesis is increased by the same factor, where $\theta = \frac{1}{2} \log \frac{1+2\beta}{1-2\beta}$. Essentially, the weights are adjusted to concentrate on difficult examples. The weights are renormalized to sum to 1, and the process repeats. After $T = O(\log(1/\varepsilon)/\beta^2)$ iterations, a majority vote among all the hypotheses achieves unweighted error most ε on the training set.

Comparing Algorithm 2 to the description of boosting given above reveals many similarities. In each iteration of Algorithm 2, prompts are given as input to a weak labeler that has quality $\beta \in (0, 1)$, where β is the edge over the trivial labeler that assigns an incorrect response to every prompt. The weight on each prompt that is correctly labeled by the previous iteration’s LLM is set to zero, and the weight on each prompt that is incorrectly labeled by the previous iteration’s LLM is increased by at least a factor $\exp(\theta)$, where $\theta = \log \frac{1}{1-\beta}$ (this fact emerges from our analysis, which proves that size of the set of prompts given to the weak labeler shrinks by a factor at least $1 - \beta$ each iteration; see Lemma 11(b) in the Appendix). As in boosting, the weights are adjusted to concentrate on difficult examples. After $T = O(\log(1/\varepsilon)/\beta)$ iterations, an LLM learned from all of the training data achieves error at most ε on the overall prompt set (see Theorem 5).

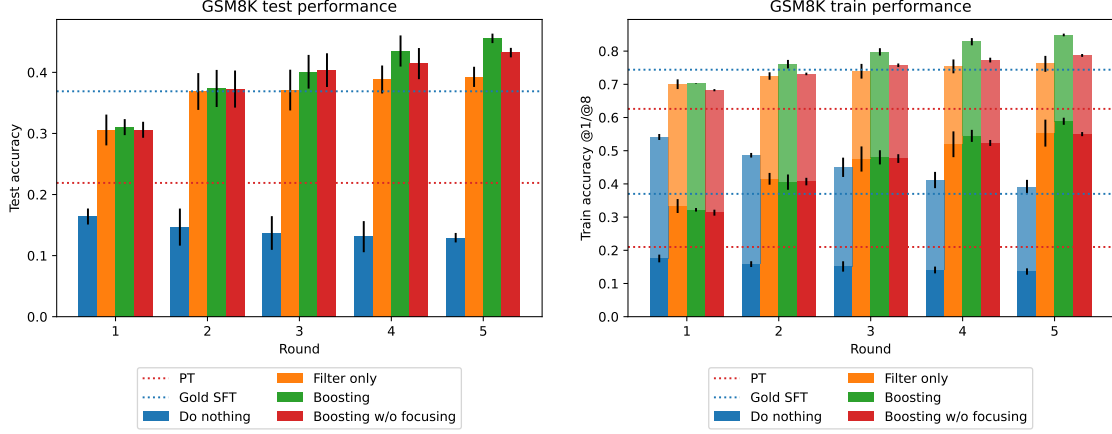


Figure 1: We plot test and train performance of our Algorithm 2 variants on GSM8K, across rounds. We report the mean and `np.std(*,ddof=1)` for 3 seeds. For train accuracy plots, we plot both train accuracy@1 (solid) and train accuracy@8 (stacked). Boosting results displayed here use *weak data* (A).

7 Experiments

Viewing Algorithm 2 as a meta-algorithm, we conduct experiments with specific instantiations using Gemma 2 2B on math problem solving [Cobbe et al., 2021, GSM8K] and Python coding [Austin et al., 2021, MBPP] tasks.¹ We select these tasks because measures of response quality here are consistent and easily verifiable.

7.1 Instantiations of Algorithm 2

Do nothing. Responses produced by the current iteration of the model are directly used as training data for the next iteration. This corresponds to setting $\alpha = 0$, omitting the **generate** operation, and using a pass-through **filter** in line 4 of Algorithm 2. This tracks the setting explored in the “model collapse” literature [Alemohammad et al., 2024, Shumailov et al., 2024, Gerstgrasser et al., 2024].

Filter only. Only correct responses in the current iteration are used for training in the next iteration. This corresponds to $\alpha = 0$ and $\gamma = 1$ in Algorithm 2. This reproduces the STaR/ReST approaches for learning from synthetic data [Zelikman et al., 2022, Gulcehre et al., 2023, Singh et al., 2024].

Boosting. The full algorithm of the present paper. In addition to the synthetic data produced by *Filter only*, we mix in weak data from the labeler. This corresponds to $\alpha > 0$ and $\beta > 0$ in Algorithm 2. We use $\alpha = 1/3$ in all experiments.

- **Boosting, w/o focusing.** We ablate out focusing on hard examples. To be precise: rather

¹Gemma models are made available under Google’s Gemma Terms of Use. GSM8K and MBPP are made available under the MIT License.